NEXT ENGINEER · _SHEERAT_

Machine Learning Techniques

BCS055

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, AIDS | **Semester:** 5 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 3rd year, Departmental Elective, 2024-25 onwards

UNIT 1 Introduction to Machine Learning

UNIT 2 Regression, Bayesian Learning and SVM

UNIT 3 Decision Trees and Instance-Based Learning

UNIT 4 Artificial Neural Networks and Deep Learning

UNIT 5 Reinforcement Learning and Genetic Algorithms

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: comparison tables, formulas and algorithm steps.
3. **Solved examples**: the numericals that appear (regression line, Naive Bayes, entropy and information gain, k-NN, perceptron, CNN sizes, Q-learning, GA crossover).
4. **Formulas to remember**: every equation you may need, in one place per unit.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Units 1 and 2. Types of learning, well-posed problems, issues in ML; then linear regression by hand, logistic regression, Bayes theorem and the Naive Bayes PlayTennis example, SVM margin and kernels.
- **Afternoon (3 hours):** Unit 3: entropy and information gain (do the PlayTennis root calculation once fully), ID3 steps, k-NN. Unit 4: perceptron rule, backpropagation formulas, CNN layers and output size.
- **Evening (2 hours):** Unit 5: MDP, Q-learning update, GA cycle with crossover and mutation. Then the "Last-hour checklist".
- **In the exam:** write the formula first, substitute the numbers neatly, and box the final answer. For algorithm questions, write numbered steps and a small diagram.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction to Machine Learning

✍️ 2-mark definitions

- **Machine learning:** the field that gives computers the ability to learn from data and improve at a task without being explicitly programmed.
- **Well-posed learning problem (Mitchell):** a program learns from experience E with respect to task T and performance measure P if its performance at T, measured by P, improves with E. Example: checkers, T = playing checkers, P = percentage of games won, E = games played against itself.
- **Supervised learning:** learning a mapping from inputs to outputs using labelled examples (classification and regression).
- **Unsupervised learning:** finding structure in unlabelled data (clustering, dimensionality reduction, association).
- **Semi-supervised learning:** learning from a small labelled set plus a large unlabelled set.
- **Reinforcement learning:** an agent learns by interacting with an environment and receiving rewards or penalties.
- **Classification vs regression:** classification predicts a discrete class label; regression predicts a continuous value.
- **Training set and test set:** data used to fit the model, and separate data used to measure how well it generalizes.
- **Overfitting:** the model fits the training data, including noise, too closely and performs poorly on new data (low bias, high variance).
- **Underfitting:** the model is too simple to capture the pattern (high bias).
- **Hypothesis space:** the set of all functions the learning algorithm may choose from.
- **Inductive bias:** the assumptions a learner uses to generalize beyond the training data.
- **Feature:** a measurable input property used by the model.

💡 Core concepts

Types of learning

Type	Data	Goal	Examples
Supervised	labelled (x, y)	predict y for new x	spam filter, house price
Unsupervised	unlabelled x	find groups or structure	customer segments (k-means), PCA
Semi-supervised	few labels, many unlabelled	use both	web page classification
Reinforcement	rewards from actions	maximize total reward	games, robots

Designing a learning system (checkers example): choose the training experience (direct or indirect feedback, who controls examples, distribution of examples), choose the target function (V: board → score), choose its representation (a linear combination of board features w0 + w1×1 + ... + w6×6),

choose the learning algorithm (LMS weight update $w_i \leftarrow w_i + \eta(V_train - \hat{V})x_i$), and build the final design (performance system, critic, generalizer, experiment generator).

Main ML approaches (overview): artificial neural networks, clustering, decision trees, Bayesian networks, support vector machines, genetic algorithms, and reinforcement learning.

Issues in machine learning

- Which algorithm works best for which problem and representation?
- How much training data is enough, and how does accuracy relate to data size?
- Noisy, missing and imbalanced data; overfitting.
- Choosing features and model complexity (bias-variance trade-off).
- Interpretability, computational cost, and ethical issues such as bias and privacy.

Data science vs ML: data science covers the whole process of collecting, cleaning, analysing and presenting data; ML is the part that builds predictive models from data.

Evaluation measures (classification). From the confusion matrix (TP, FP, TN, FN): accuracy = (TP + TN)/total, precision = TP/(TP + FP), recall = TP/(TP + FN), F1 = 2PR/(P + R).

🧮 Solved examples

Example 1 (well-posed problems).

- Handwriting recognition: T = recognising handwritten words, P = percentage of words correctly classified, E = a database of labelled handwritten words.
- Self-driving: T = driving on public roads using sensors, P = average distance travelled before an error, E = recorded driving by a human driver.

Example 2 (confusion matrix). TP = 40, FP = 10, FN = 5, TN = 45.

- Accuracy = 85/100 = 0.85.
- Precision = 40/50 = 0.80.
- Recall = 40/45 ≈ 0.889.
- F1 = 2(0.80)(0.889)/(1.689) ≈ 0.842.

Formulas to remember

- LMS update: $w_i \leftarrow w_i + \eta\,(V_train(b) - \hat{V}(b))\,x_i$.
- Accuracy, precision, recall, F1 as above.
- Error rate = 1 − accuracy.

⭐ Important questions

1. Define machine learning and a well-posed learning problem with examples.
2. Explain supervised, unsupervised and reinforcement learning with examples and a comparison table.
3. Explain the steps in designing a learning system (checkers).

NEXT ENGINEER _sheerat_

4. Give an overview of the main ML approaches (ANN, clustering, decision tree, Bayesian network, SVM, GA).
5. Discuss the issues in machine learning; explain overfitting and the bias-variance trade-off; compare data science and ML.

NEXT ENGINEER
sheerat

UNIT 2 Regression, Bayesian Learning and SVM

✍️ 2-mark definitions

- **Linear regression:** models the output as a linear function of the inputs, y = a + bx, chosen to minimize the sum of squared errors.
- **Logistic regression:** a classification model that passes a linear combination of inputs through the sigmoid σ(z) = 1/(1 + e^(−z)) to get a probability between 0 and 1.
- **Bayes theorem:** $P(h \mid D) = P(D \mid h)\, P(h) / P(D)$.
- **Prior and posterior:** the prior P(h) is the belief before seeing data; the posterior $P(h \mid D)$ is the belief after.
- **MAP hypothesis:** the hypothesis with maximum posterior: $h_{MAP} = \text{argmax}\, P(D \mid h)\, P(h)$. With equal priors it is the maximum likelihood (ML) hypothesis.
- **Bayes optimal classifier:** classifies by combining the predictions of all hypotheses weighted by their posteriors; no other classifier using the same hypothesis space and prior does better on average.
- **Naive Bayes classifier:** assumes features are conditionally independent given the class: $v_{NB} = \text{argmax}\, P(v) \prod P(a_i \mid v)$.
- **Bayesian belief network:** a directed acyclic graph whose nodes are random variables and whose edges show dependencies; each node has a conditional probability table.
- **EM algorithm:** Expectation-Maximization, an iterative method for maximum likelihood when some data is hidden or missing; E-step estimates hidden values, M-step re-estimates parameters.
- **Support vector machine (SVM):** a classifier that finds the hyperplane with the maximum margin between two classes.
- **Support vectors:** the training points closest to the hyperplane; they alone define it.
- **Kernel:** a function $K(x, y) = \varphi(x) \cdot \varphi(y)$ that computes a dot product in a higher-dimensional space without mapping explicitly (the kernel trick).

💡 Core concepts

Linear vs logistic regression

Point	Linear	Logistic
Task	regression	classification
Output	any real number	probability 0 to 1
Function	$y = w \cdot x + b$	$\sigma(w \cdot x + b)$
Loss	squared error	log loss (cross-entropy)
Fitting	least squares (closed form)	gradient descent

SVM key ideas

- Hyperplane: $w \cdot x + b = 0$. Margin = $2/\|w\|$.
- Hard margin: minimize $\frac{1}{2}\|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$.
- Soft margin: adds slack variables ξ_i and penalty C to allow some errors.
- Kernels for non-linear data: linear $K = x \cdot y$; polynomial $K = (x \cdot y + c)^d$; Gaussian RBF $K = \exp(-\|x - y\|^2/2\sigma^2)$; sigmoid $K = \tanh(\alpha x \cdot y + c)$.
- Properties: effective in high dimensions, uses only support vectors (memory efficient), but slow on very large data and needs kernel choice.

EM algorithm (e.g. for a mixture of two Gaussians)

1. Start with initial guesses for the means.
2. E-step: for each point, compute the probability that it came from each Gaussian (expected hidden values).
3. M-step: recompute each mean as the weighted average of the points.
4. Repeat until the parameters stop changing (converges to a local maximum of likelihood).

Bayesian network example: Burglary → Alarm ← Earthquake; Alarm → JohnCalls, Alarm → MaryCalls. Joint probability = product of each node's probability given its parents: $P(J, M, A, B, E) = P(J \mid A)\, P(M \mid A)\, P(A \mid B, E)\, P(B)\, P(E)$.

🧮 Solved examples

Example 1 (linear regression by least squares). Points (1, 2), (2, 4), (3, 5), (4, 4), (5, 5).

- $\bar{x} = 3$, $\bar{y} = 4$.
- $\Sigma(x - \bar{x})(y - \bar{y}) = (-2)(-2) + (-1)(0) + 0 + (1)(0) + (2)(1) = 6$.
- $\Sigma(x - \bar{x})^2 = 4 + 1 + 0 + 1 + 4 = 10$.
- $b = 6/10 = 0.6$; $a = \bar{y} - b\,\bar{x} = 4 - 1.8 = 2.2$.
- Line: **$y = 2.2 + 0.6x$**. Prediction at $x = 6$: 5.8.

Example 2 (Bayes theorem). A disease affects 1% of people. The test detects it 90% of the time and gives false positives 5% of the time. $P(\text{disease} \mid \text{positive}) = (0.9 \times 0.01)/(0.9 \times 0.01 + 0.05 \times 0.99) = 0.009/0.0585 \approx$ **0.154**.

Example 3 (Naive Bayes, PlayTennis). Classify (Outlook = sunny, Temp = cool, Humidity = high, Wind = strong) using the standard 14-day PlayTennis data (9 yes, 5 no).

- Yes: $P(\text{yes}) = 9/14$; $P(\text{sunny} \mid \text{yes}) = 2/9$; $P(\text{cool} \mid \text{yes}) = 3/9$; $P(\text{high} \mid \text{yes}) = 3/9$; $P(\text{strong} \mid \text{yes}) = 3/9$. Product ≈ 0.0053.
- No: $P(\text{no}) = 5/14$; $P(\text{sunny} \mid \text{no}) = 3/5$; $P(\text{cool} \mid \text{no}) = 1/5$; $P(\text{high} \mid \text{no}) = 4/5$; $P(\text{strong} \mid \text{no}) = 3/5$. Product ≈ 0.0206.
- 0.0206 > 0.0053, so the answer is **PlayTennis = No** (normalized probability of no ≈ 0.795).

Example 4 (logistic output). $w \cdot x + b = 2$ gives $\sigma(2) = 1/(1 + e^{-2}) \approx 0.88$, so the predicted class is 1 (threshold 0.5).

Formulas to remember

- $b = \Sigma(x - \bar{x})(y - \bar{y})/\Sigma(x - \bar{x})^2$, $a = \bar{y} - b\,\bar{x}$.

- $\sigma(z) = 1/(1 + e^{(-z)})$; log loss $= -[y \log p + (1 - y) \log(1 - p)]$.
- Bayes: $P(h \mid D) = P(D \mid h)\, P(h)/P(D)$.
- Naive Bayes: $v = \text{argmax}\, P(v) \prod P(a_i \mid v)$.
- SVM margin $= 2/\|w\|$.

⭐ Important questions

1. Explain linear regression and fit a line to given data; compare linear and logistic regression.
2. State Bayes theorem; explain MAP, ML and the Bayes optimal classifier.
3. Explain the Naive Bayes classifier and solve a classification example.
4. Explain Bayesian belief networks and the EM algorithm.
5. Explain SVM: hyperplane, margin, support vectors, kernels (linear, polynomial, Gaussian), properties and issues.

NEXT ENGINEER _sheerat_

UNIT 3 Decision Trees and Instance-Based Learning

✍️ 2-mark definitions

- **Decision tree:** a tree where internal nodes test attributes, branches are attribute values and leaves are class labels.
- **Entropy:** a measure of impurity of a set S: Entropy(S) = $-\Sigma\ p_i \log_2 p_i$. It is 0 for a pure set and 1 for a 50-50 two-class set.
- **Information gain:** the expected reduction in entropy from splitting S on attribute A: Gain(S, A) = Entropy(S) − Σ ($|Sv|/|S|$) Entropy(Sv).
- **ID3:** a top-down, greedy algorithm that builds a decision tree by choosing the attribute with the highest information gain at each node.
- **Inductive bias of ID3:** prefers shorter trees, with high-gain attributes near the root (Occam's razor).
- **Pruning:** removing branches that do not improve accuracy on validation data, to reduce overfitting (reduced-error pruning, rule post-pruning).
- **Instance-based (lazy) learning:** stores training examples and postpones generalization until a query arrives, e.g. k-NN.
- **k-nearest neighbour (k-NN):** classifies a query by the majority class among its k closest training examples (or averages their values for regression).
- **Locally weighted regression:** fits a separate local model around each query point, weighting nearby points more.
- **Radial basis function (RBF) network:** a neural network whose hidden units use Gaussian-like functions of distance from a centre, followed by a linear output layer.
- **Case-based reasoning (CBR):** solves new problems by retrieving and adapting solutions of similar past cases, stored as rich symbolic descriptions.

💡 Core concepts

ID3 algorithm

1. If all examples have the same class, return a leaf with that class.
2. If no attributes remain, return a leaf with the majority class.
3. Otherwise pick the attribute A with the highest information gain as the node.
4. For each value v of A, create a branch with the subset Sv and recurse (if Sv is empty, use the majority class).

Issues in decision tree learning: overfitting (fix with pruning), continuous attributes (choose thresholds), attributes with many values (use gain ratio), missing values (use most common value or fractional examples), and attributes with costs.

Eager vs lazy learning

Point	Eager (decision tree, SVM, ANN)	Lazy (k-NN, CBR)
Training	builds a model in advance	just stores data
Prediction	fast	slow (searches stored data)
Generalization	global	local, per query
Memory	model only	all training data

k-NN details: distance is usually Euclidean $\sqrt{\Sigma(x_i - y_i)^2}$. Normalize features first. Choose odd k for two classes. Distance-weighted k-NN gives closer neighbours more weight ($w_i = 1/d^2$). Small k is sensitive to noise; large k blurs class boundaries.

🧮 Solved examples

Example 1 (entropy). S has 9 positive and 5 negative examples. Entropy(S) = $-(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$ = **0.940**.

Example 2 (information gain, PlayTennis root).

- Wind: weak has 6+, 2− (entropy 0.811); strong has 3+, 3− (entropy 1.0). Gain = 0.940 − (8/14)(0.811) − (6/14)(1.0) = **0.048**.
- Humidity: high 3+, 4− (0.985); normal 6+, 1− (0.592). Gain = 0.940 − 0.5(0.985) − 0.5(0.592) = **0.151**.
- Outlook: sunny 2+, 3− (0.971); overcast 4+, 0− (0); rain 3+, 2− (0.971). Gain = 0.940 − (5/14)(0.971) − 0 − (5/14)(0.971) = **0.246**.
- Temperature gives 0.029.
- **Outlook** has the highest gain, so it is the root. The overcast branch becomes a "Yes" leaf; sunny is next split on Humidity, and rain on Wind.

Example 3 (k-NN). Training points: A(1, 1) red, B(2, 1) red, C(4, 3) blue, D(6, 5) blue, E(3, 2) red. Query Q(3, 3), k = 3.

- Distances from Q: A $\sqrt{8} \approx 2.83$, B $\sqrt{5} \approx 2.24$, C 1, D $\sqrt{13} \approx 3.61$, E 1.
- Three nearest: C (blue, 1), E (red, 1), B (red, 2.24).
- Votes: red 2, blue 1, so Q is classified **red**.
- With k = 1 there is a tie between C and E at distance 1; this is why odd k and a tie-breaking rule (such as distance weighting) matter.

Formulas to remember

- Entropy(S) = $-\Sigma\, p_i \log_2 p_i$.
- Gain(S, A) = Entropy(S) − Σ ($|Sv|/|S|$) Entropy(Sv).
- Gain ratio = Gain/SplitInfo, SplitInfo = $-\Sigma$ ($|Sv|/|S|$) $\log_2(|Sv|/|S|)$.
- Euclidean distance $\sqrt{\Sigma(x_i - y_i)^2}$; weight $w_i = 1/d(xq, x_i)^2$.
- RBF unit: $\varphi(x) = \exp(-\|x - c\|^2/2\sigma^2)$.

NEXT ENGINEER _sheerat_

⭐ Important questions

1. Explain decision tree learning and the ID3 algorithm; build a tree using entropy and information gain.
2. Explain inductive bias and issues in decision tree learning (overfitting, pruning, continuous and missing values).
3. Explain instance-based learning and the k-NN algorithm with an example.
4. Explain locally weighted regression and radial basis function networks.
5. Explain case-based learning; compare eager and lazy learning.

NEXT ENGINEER
sheerat

UNIT 4 Artificial Neural Networks and Deep Learning

✍️ 2-mark definitions

- **Artificial neural network (ANN):** a network of simple processing units (neurons) connected by weighted links, inspired by the brain, that learns by adjusting weights.
- **Perceptron:** a single neuron that outputs 1 if $w \cdot x + b > 0$, else 0 (or −1); it can learn only linearly separable functions.
- **Activation function:** a function applied to a neuron's weighted sum: step, sigmoid, tanh, ReLU, softmax.
- **Multilayer perceptron (MLP):** a feed-forward network with one or more hidden layers that can learn non-linear functions such as XOR.
- **Gradient descent:** updates weights in the direction that reduces error: $w \leftarrow w - \eta\, \partial E/\partial w$. The delta rule is gradient descent for a linear unit.
- **Backpropagation:** an algorithm that computes error gradients for all weights by propagating errors backward from the output layer.
- **Learning rate (η):** the step size of each weight update.
- **Epoch:** one complete pass through the training data.
- **Self-organizing map (SOM):** an unsupervised network (Kohonen) that maps high-dimensional input onto a low-dimensional grid while keeping neighbourhood relations, using competitive learning.
- **Deep learning:** machine learning with neural networks of many layers that learn features automatically.
- **Convolutional neural network (CNN):** a deep network for grid data (images) using convolution layers, pooling layers and fully connected layers.
- **Pooling:** down-sampling a feature map, e.g. max pooling takes the maximum in each window.

💡 Core concepts

Activation functions

Function	Formula	Range	Note
Step	1 if $z \geq 0$ else 0	{0, 1}	perceptron
Sigmoid	$1/(1 + e^{-z})$	(0, 1)	derivative $\sigma(1 - \sigma)$; vanishing gradient
tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	(−1, 1)	zero-centred
ReLU	$\max(0, z)$	$[0, \infty)$	fast, common in deep nets
Softmax	$e^{z_i}/\Sigma\, e^{z_j}$	(0, 1), sum 1	multi-class output

Perceptron training rule: $w_i \leftarrow w_i + \eta(t - o)x_i$, where t is the target and o the output. It converges if the data is linearly separable (perceptron convergence theorem). XOR is not linearly separable, so one perceptron cannot learn it.

Backpropagation algorithm (sigmoid units)

1. Initialize weights with small random values.
2. Forward pass: compute the output of every unit.
3. Output unit k: $\delta k = ok(1 - ok)(tk - ok)$.
4. Hidden unit h: $\delta h = oh(1 - oh) \Sigma w(k,h) \delta k$.
5. Update every weight: $w(j,i) \leftarrow w(j,i) + \eta \delta j x(j,i)$.
6. Repeat for all examples and epochs until the error is small.

Adding a momentum term $\alpha \Delta w(n - 1)$ speeds learning and helps escape small local minima.

CNN architecture: input image → convolution (filters slide over the image and create feature maps) → ReLU → pooling → repeated blocks → flatten → fully connected layers → softmax. Advantages: shared weights (fewer parameters), local connectivity, translation invariance. Uses: image classification, object detection, face recognition.

Deep learning vs traditional ML: deep learning learns features automatically, needs large data and GPUs, and scales well with data; traditional ML needs hand-crafted features and works with less data.

SOM algorithm: initialize weight vectors of grid nodes; for each input find the best matching unit (smallest distance); update it and its neighbours: $w \leftarrow w + \eta h(t)(x - w)$; shrink the neighbourhood and learning rate over time.

🧮 Solved examples

Example 1 (perceptron for AND). Weights w1 = 1, w2 = 1, bias b = −1.5 with step output:

- (0, 0): −1.5 → 0; (0, 1): −0.5 → 0; (1, 0): −0.5 → 0; (1, 1): 0.5 → 1. This matches AND.
- For OR, b = −0.5 works. No single line separates XOR's classes.

Example 2 (one perceptron update). $w = (0.2, -0.1)$, $b = 0$, $\eta = 0.1$, input $x = (1, 1)$, target $t = 1$. Sum = 0.1 > 0 so $o = 1$; $t - o = 0$, no change. For $x = (1, 0)$ with $t = 0$: sum = 0.2, $o = 1$, error −1, so $w1 = 0.2 - 0.1 = 0.1$, $b = -0.1$.

Example 3 (backprop output delta). Output $o = 0.7$, target $t = 1$: $\delta = 0.7 \times 0.3 \times 0.3 = 0.063$. With $\eta = 0.5$ and input to that weight 0.6: $\Delta w = 0.5 \times 0.063 \times 0.6 \approx 0.019$.

Example 4 (CNN output size). Output width = $(W - F + 2P)/S + 1$.

- 32 × 32 image, 5 × 5 filter, P = 0, S = 1: $(32 - 5)/1 + 1 =$ **28**, so 28 × 28 per filter.
- Then 2 × 2 max pooling with stride 2: **14 × 14**.
- With 6 filters of size 5 × 5 × 3, parameters = $6 \times (75 + 1) =$ **456**.

Formulas to remember

- Perceptron: $w_i \leftarrow w_i + \eta(t - o)x_i$.
- Delta rule: $\Delta w_i = \eta \Sigma(t - o)x_i$.
- Output $\delta = o(1 - o)(t - o)$; hidden $\delta = o(1 - o) \Sigma w \delta$.
- CNN size: $(W - F + 2P)/S + 1$.
- Sigmoid derivative: $\sigma(z)(1 - \sigma(z))$.

NEXT ENGINEER _sheerat_

⭐ Important questions

1. Explain the perceptron and its training rule; why can it not learn XOR?
2. Explain multilayer networks and derive or state the backpropagation algorithm.
3. Explain activation functions and the role of learning rate and momentum.
4. Explain self-organizing maps.
5. Explain deep learning and the architecture of a CNN with convolution and pooling; mention applications.

NEXT ENGINEER
sheerat

UNIT 5 Reinforcement Learning and Genetic Algorithms

✍️ 2-mark definitions

- **Reinforcement learning (RL):** learning what actions to take in each state to maximize cumulative reward through trial and error.
- **Agent and environment:** the agent chooses actions; the environment returns the next state and a reward.
- **Markov decision process (MDP):** a model (S, A, P, R, γ) of states, actions, transition probabilities, rewards and a discount factor, where the next state depends only on the current state and action.
- **Policy (π):** a mapping from states to actions.
- **Value function V(s):** expected discounted return starting from s and following π. **Q(s, a):** expected return after taking a in s and then following π.
- **Discount factor (γ):** $0 \le \gamma < 1$; it weighs immediate rewards more than future ones.
- **Exploration vs exploitation:** trying new actions to learn more versus choosing the best known action (ε-greedy balances them).
- **Q-learning:** a model-free, off-policy RL algorithm that learns Q-values from experience.
- **Deep Q-learning (DQN):** Q-learning that uses a deep neural network to approximate Q(s, a), with experience replay and a target network.
- **Genetic algorithm (GA):** a search method inspired by natural evolution that evolves a population of candidate solutions using selection, crossover and mutation.
- **Chromosome and gene:** a chromosome encodes one candidate solution (e.g. a bit string); genes are its positions.
- **Fitness function:** a function that scores how good a chromosome is.
- **Crossover:** combining parts of two parents to create offspring. **Mutation:** randomly changing a gene to keep diversity.

💡 Core concepts

RL vs supervised learning: RL has no labelled correct actions, feedback is a delayed reward, data depends on the agent's own actions, and the agent must balance exploration and exploitation.

Q-learning algorithm

1. Initialize Q(s, a) arbitrarily (e.g. 0).
2. Observe the current state s.
3. Choose action a (ε-greedy), execute it, get reward r and next state s′.
4. Update: $Q(s, a) \leftarrow Q(s, a) + \alpha\,[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$.
5. Set $s \leftarrow s'$ and repeat until the episode ends; run many episodes.

For deterministic environments with $\alpha = 1$: $Q(s, a) = r + \gamma \max Q(s', a')$.

Deep Q-learning: a network takes the state and outputs Q for every action; transitions are stored in a replay buffer and sampled randomly; a separate target network gives stable targets r + γ max

Q_target(s′, a′).

Applications of RL: game playing (chess, Go, Atari), robotics, self-driving, recommendation, resource management.

Genetic algorithm cycle (generational cycle)

1. Encode solutions as chromosomes (binary, real-valued, permutation, tree encodings).
2. Create a random initial population.
3. Evaluate fitness of each chromosome.
4. Select parents (roulette wheel, tournament, rank, elitism).
5. Apply crossover (single-point, two-point, uniform) and mutation (bit flip, swap).
6. Form the new generation; repeat until a stopping condition (generations or fitness reached).

Applications of GA: optimization, scheduling and timetabling, TSP, feature selection, neural network weight tuning, design problems.

GA vs traditional search: GA works on a population, uses fitness only (no derivatives), uses probabilistic rules, and works on encoded solutions rather than parameters directly.

🧮 Solved examples

Example 1 (Q-learning update). Q(s, a) = 0, reward r = 10, α = 0.5, γ = 0.9, and max Q(s′, ·) = 20. New Q = 0 + 0.5 × (10 + 0.9 × 20 − 0) = 0.5 × 28 = **14**.

Example 2 (discounted return). Rewards 1, 1, 1, 10 with γ = 0.9: G = 1 + 0.9 + 0.81 + 10 × 0.729 = **10.0** (1 + 0.9 + 0.81 + 7.29).

Example 3 (roulette-wheel selection). Fitness values 10, 20, 30, 40 (total 100). Selection probabilities 0.1, 0.2, 0.3, 0.4. The fittest chromosome is chosen 40% of the time.

Example 4 (crossover and mutation). Parents 11010 and 01101, single-point crossover after bit 2:

- Child 1 = 11 | 101 = **11101**.
- Child 2 = 01 | 010 = **01010**.
- Mutating bit 5 of child 1 gives 11100.

Example 5 (GA for maximizing $f(x) = x^2$ on 0 to 31). Encode x in 5 bits. Initial strings 01101 (13, f = 169), 11000 (24, f = 576), 01000 (8, f = 64), 10011 (19, f = 361). Total 1170; 11000 has the highest selection probability (0.49). Crossing 11000 with 10011 after bit 3 gives 11011 (27, f = 729), better than both parents.

Formulas to remember

- Return G = Σ γ^k r(t + k + 1).
- Q-learning: Q(s, a) ← Q(s, a) + α [r + γ max Q(s′, a′) − Q(s, a)].
- Bellman: V(s) = max over a of [R(s, a) + γ Σ P(s′ | s, a) V(s′)].
- Roulette selection probability $p_i = f_i / \Sigma f$.

⭐ Important questions

1. Explain reinforcement learning, its elements and how it differs from supervised learning.
2. Explain the Markov decision process and Bellman equation.
3. Explain the Q-learning algorithm with an example; explain deep Q-learning.
4. Explain the genetic algorithm cycle: encoding, selection, crossover, mutation and fitness.
5. Discuss applications of RL and GA; solve a small GA problem for one generation.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ Well-posed problem (T, P, E) with checkers; types of learning table; issues in ML.
- ☐ Confusion matrix: accuracy, precision, recall, F1.
- ☐ Least squares b and a; y = 2.2 + 0.6x example; sigmoid and log loss.
- ☐ Bayes theorem; MAP vs ML; Naive Bayes PlayTennis answer "No".
- ☐ Bayesian network joint probability; EM steps.
- ☐ SVM margin $2/\|w\|$, support vectors, kernels.
- ☐ Entropy 0.940; gains Outlook 0.246, Humidity 0.151, Wind 0.048, Temp 0.029; ID3 steps.
- ☐ k-NN and distance weighting; RBF; case-based reasoning; eager vs lazy.
- ☐ Perceptron rule; AND weights; XOR problem; backprop deltas.
- ☐ Activation table; CNN size formula; SOM update.
- ☐ MDP tuple; Q-learning update (14 example); DQN replay and target network.
- ☐ GA cycle; roulette wheel; single-point crossover 11101 / 01010.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Machine Learning	Tom M. Mitchell	McGraw-Hill Education (India)	WorldCat
2	Introduction to Machine Learning	Ethem Alpaydin	MIT Press	WorldCat
3	Machine Learning: An Algorithmic Perspective	Stephen Marsland	CRC Press	WorldCat
4	Pattern Recognition and Machine Learning	Christopher M. Bishop	Springer	WorldCat

Syllabus source: AKTU B.Tech 3rd year syllabus, 2024-25 (copy hosted by an affiliated college, PDF)